

Intro to Deep Learning

Lecture 13: LLM Adaptation

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. What does pretraining teach a GPT-style model?
2. What does supervised fine-tuning change?
3. Why does prompt formatting matter?
4. What is LoRA useful for?
5. What is the high-level idea of RLHF?
6. What can a tiny instruction dataset teach, and what can it not teach?

Lecture rhythm: base LM -> SFT -> label masking -> LoRA -> preferences

Recap From Lecture 11

GPT-style models are pretrained with next-token prediction:

```
predict the next token from previous tokens
```

This teaches broad language modeling ability.

But a base pretrained model is not automatically:

- helpful
- polite
- instruction-following
- safe

Adaptation shapes behavior.

Part 1

From base LM to assistant behavior

Base LM vs Assistant-Like LM

Base language model:

continues text according to pretraining distribution

Instruction-following model:

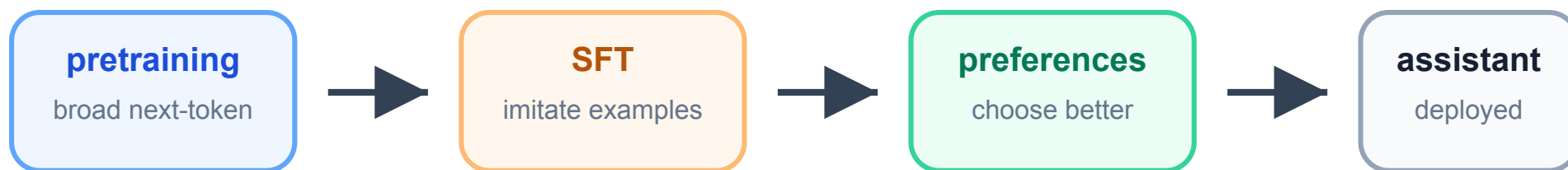
responds to user instructions in a useful format

The architecture may be similar.

The difference is largely in adaptation data and training objectives.

LLM Adaptation Stack

Adaptation turns a base language model into task-following behavior



Different stages use different data and optimize different behavior.

What Pretraining Teaches

Pretraining can teach:

- language patterns
- syntax and style
- common facts
- text continuation
- broad representations
- some reasoning-like patterns

It is broad and general.

It is not targeted to one narrow classroom task.

What Pretraining Does Not Guarantee

Pretraining does not guarantee:

- following a specific prompt format
- answering concisely
- being polite
- refusing unsafe requests
- optimizing for user preferences
- producing exactly the target style

So we often adapt the model after pretraining.

Part 2

Supervised fine-tuning

Supervised Fine-Tuning

Supervised fine-tuning means training on examples like:

```
input prompt -> desired response
```

For instruction tuning:

```
instruction + optional input -> response
```

The model learns:

```
when the prompt looks like this,  
produce a response like that
```

Example Instruction-Tuning Task

Polite rewriting is one example of instruction tuning.

Each example contains:

```
instruction
input
response
```

The goal is narrow:

```
rewrite a message to sound polite and professional
while preserving intent
```

This is behavior shaping, not broad knowledge learning.

Polite Rewriting Example

SFT data pairs a prompt with a desired response

instruction

Rewrite politely
and professionally.

input

Send me the report.
You are late.

response

Could you please send
the report when you
have a chance?

Prompt Formatting

We convert each example into one text sequence.

Example format:

```
### Instruction:  
<instruction>
```

```
### Input:  
<input>
```

```
### Response:  
<response>
```

The model learns both the task and the expected format.

Why Format Matters

The format tells the model:

- where the instruction starts
- where the input starts
- where the answer should begin
- what response style is expected

If training and inference formats differ, performance can drop.

Instruction tuning is partly about teaching this interface.

Prompt Template

Prompt format is part of the interface the model learns

Instruction: task description

Input: user text to transform

Response: target output begins here

Changing these boundaries between training and inference can hurt behavior.

Causal LM Objective For SFT

The model is still a causal language model.

It predicts the next token in the sequence.

For SFT, the useful part is:

```
predict response tokens  
given instruction, input, and previous response tokens
```

So the same next-token machinery becomes task adaptation.

SFT Loss

For response tokens:

$$L = - \sum_t \log p(y_t \mid \textit{prompt}, y_{<t})$$

Meaning:

make the desired response tokens more likely
given the prompt

This is teacher forcing:

train using the correct previous response tokens

Label Masking

For response-only SFT, prompt tokens are masked out in the labels.

Why?

```
we do not mainly train the model to copy the prompt  
we train it to produce the response
```

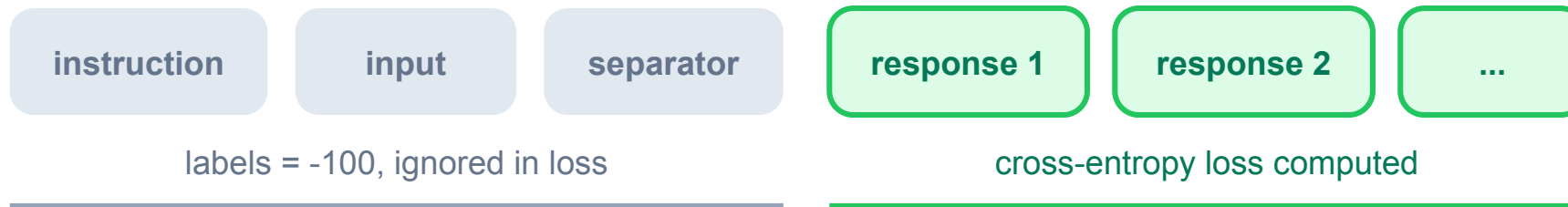
Conceptually:

```
prompt tokens:  ignore in loss  
response tokens: compute loss
```

This focuses learning on the desired answer behavior.

Label Masking

Response-only loss focuses learning on the answer behavior



Before vs After SFT

Before SFT:

the base model may continue the prompt instead of solving the requested task

After SFT:

the model should better follow the trained instruction-response pattern

The important comparison is behavioral:

- task alignment
- output format
- response quality for the target task
- generalization to held-out prompts

Before vs After SFT: Picture

SFT changes behavior for a prompt format and task

before SFT

may continue the prompt
may ignore the desired style
may produce inconsistent format

after SFT

more likely to answer
uses polite rewrite style
matches trained template better

Mini-Check: Strange Copying

Training loss decreases, but generations mostly copy the prompt template.

You inspect labels and see prompt tokens are not set to `-100`.

What is the model being trained to do?

What should be masked?

Answer: Strange Copying

The model is being trained to predict the prompt/template tokens too.

That can make the loss look good while the model learns to reproduce:

```
### Instruction:  
### Input:  
### Response:
```

For response-only SFT:

```
prompt labels    -> -100  
response labels -> real token ids
```

Tiny Dataset, Visible Shift

A tiny dataset can change a narrow behavior.

Why?

the base model already knows English
SFT only nudges task behavior, style, and format

It does not need to learn language from scratch.

It needs to learn:

for this prompt format, produce this kind of response

What Tiny SFT Cannot Teach

A small instruction dataset will not reliably teach:

- broad world knowledge
- robust reasoning
- safety behavior across domains
- all possible writing styles
- perfect instruction following
- deep factual accuracy

It can overfit to examples and prompt patterns.

Keep expectations realistic.

Mini-Check: Bad Held-Out Behavior

Training examples use:

```
### Instruction / ### Input / ### Response
```

At inference, you test:

```
Can you make this message nicer?
```

The model performs much worse.

What is the likely issue, and what should you test next?

Answer: Bad Held-Out Behavior

The likely issue is a train-test prompt format mismatch.

The model learned a narrow interface:

```
### Instruction / ### Input / ### Response
```

Test first with the same format used during training.

Then test realistic user prompts separately to measure generalization.

Generalization Questions

After SFT, test held-out prompts.

Ask:

- does the response follow the format?
- does it solve the requested task?
- does it preserve the important input meaning?
- is the response quality acceptable?
- does it work on wording not seen in training?

This is more informative than only watching training loss.

Full Fine-Tuning Cost

Full fine-tuning updates many parameters.

For large LMs, this can be expensive:

- high GPU memory
- slow training
- large checkpoints
- careful infrastructure
- risk of overfitting or forgetting

This motivates parameter-efficient fine-tuning.

Part 3

LoRA and parameter-efficient adaptation

LoRA Idea

LoRA freezes the original weight matrix and learns a small update.

Instead of directly changing:

W

learn:

$$W' = W + BA$$

where A and B are low-rank trainable matrices.

The original W stays frozen.

Why LoRA Helps

LoRA trains far fewer parameters.

Benefits:

- lower memory use
- smaller saved adapters
- faster experimentation
- easier to keep multiple task adaptations
- less invasive than full fine-tuning

It is widely used for adapting large models.

It is often used when full fine-tuning is too expensive.

LoRA Update

LoRA freezes W and learns a small low-rank update



SFT Is Not Preference Alignment

SFT teaches the model to imitate desired responses.

But real assistants often need preference decisions:

which of two plausible answers is better?

Examples:

- more helpful
- more honest
- safer
- better formatted
- less verbose

This motivates preference-based alignment methods.

Part 4

Preference alignment

RLHF: High-Level Idea

RLHF stands for:

Reinforcement Learning from Human Feedback

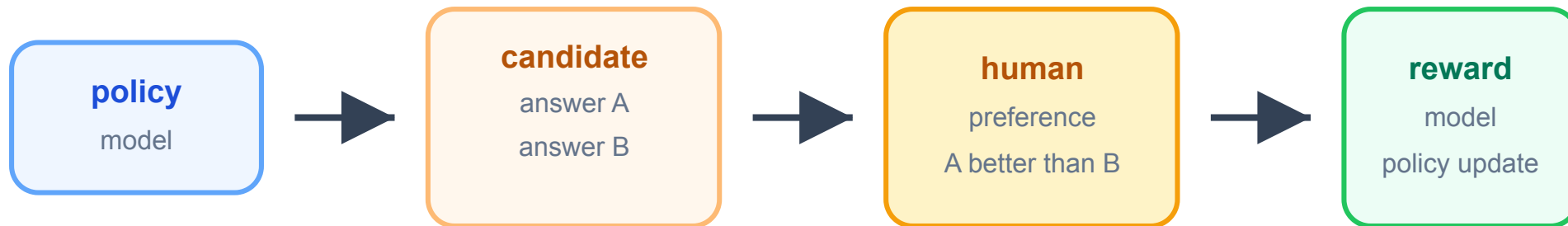
High-level process:

model produces candidate answers
humans rank or compare them
train a reward model from preferences
update the policy model toward higher reward

We only need the block diagram level.

RLHF Pipeline

RLHF uses preferences, not only ideal demonstrations



At this level: learn what humans prefer, then push the model toward those outputs.

How The Reward Model Is Trained

Reward-model data usually contains comparisons:

```
prompt  
chosen response  
rejected response
```

The reward model reads:

```
(prompt, response) -> scalar reward score
```

It is trained so:

$$r_{\theta}(x, y_{\text{chosen}}) > r_{\theta}(x, y_{\text{rejected}})$$

Reward Model Loss

A common pairwise ranking objective is:

$$L_{\text{RM}} = -\log \sigma(r_{\theta}(x, y_{\text{chosen}}) - r_{\theta}(x, y_{\text{rejected}}))$$

Intuition:

increase the score gap between the preferred and rejected answers

The reward model learns human preference patterns, not ground-truth labels.

How The Reward Model Is Used

After reward-model training:

1. start from an SFT model
2. generate candidate responses
3. score them with the reward model
4. update the policy toward higher reward
5. keep it close to the SFT/reference model

The last step is important:

$$\text{objective} \approx \text{reward} - \beta D_{\text{KL}}(\pi_{\text{policy}} \parallel \pi_{\text{ref}})$$

Reward Model Is Not The Assistant

The reward model is a learned evaluator.

It does not generate the final answer.

The policy model generates answers, and the reward model gives training feedback.

Why this matters:

- reward models can be wrong
- policy models can exploit reward-model weaknesses
- KL regularization helps avoid drifting too far from the SFT model
- final evaluation still needs human or task-based checks

SFT vs RLHF

SFT data:

```
prompt -> ideal response
```

Preference data:

```
prompt -> response A vs response B -> which is better?
```

SFT teaches:

```
imitate this response style
```

RLHF-style alignment teaches:

```
prefer outputs humans rate more highly
```

Mini-Check: Preference Data

You have examples like:

```
prompt, answer A, answer B, human prefers B
```

You train with ordinary SFT as if `answer B` is the only target response.

What information is being discarded?

Why is this not the same as preference alignment?

Answer: Preference Data

Ordinary SFT keeps only:

```
prompt -> answer B
```

It discards the comparison:

```
answer B was preferred over answer A
```

Preference alignment uses the relative signal:

```
score(prompt, B) > score(prompt, A)
```

That teaches the model what kinds of differences humans prefer.

Failure Modes

Adapted LMs can still fail.

Common issues:

- hallucination
- memorization
- prompt sensitivity
- toxic or biased outputs
- overfitting to format
- verbosity or refusal mistakes
- brittle behavior outside training distribution

SFT improves behavior, but does not solve everything.

Prompt Sensitivity

Small instruction-tuned models can be very sensitive to format.

Example:

```
trained with "### Instruction / ### Input / ### Response"  
tested with a plain natural-language prompt
```

The model may behave worse.

So evaluate with both:

- training-like prompts
- realistic user prompts

Practical Constraints

For classroom fine-tuning:

- use a small model
- use a small dataset
- keep sequence lengths modest
- compare before vs after
- focus on qualitative behavior
- avoid overclaiming quality

The goal is understanding, not building a production assistant.

Common Mistakes

Mistake 1:

training on prompt tokens when you intended response-only loss

Mistake 2:

changing prompt format between training and inference

Mistake 3:

expecting tiny SFT to add broad knowledge

Mistake 4:

judging only by loss, not generated behavior

Debugging Checklist

Before training, check:

- dataset examples look correct
- prompt template is consistent
- tokenizer has padding configured
- labels mask prompt tokens correctly
- generated outputs are compared before and after SFT
- held-out prompts are tested
- claims match the tiny dataset scale

Practical Workflow

A typical adaptation workflow:

1. inspect a polite-rewriting instruction dataset
2. format prompt-response examples
3. load a small causal language model
4. compare generations before SFT
5. mask prompt labels and train on response tokens
6. compare generations after SFT
7. test held-out prompts
8. reflect on what SFT changed

Takeaways

LLM adaptation turns a base model into a task-following model.

Key ideas:

- pretraining teaches broad next-token prediction
- SFT teaches desired prompt-response behavior
- prompt format is part of the learned interface
- LoRA adapts models with small trainable updates
- RLHF uses preferences to align outputs
- tiny datasets can shape narrow behavior, not broad capability

Next lecture:

CLIP and multimodal representation learning